

TEMA I

DESARROLLO DE APLICACIONES BACK EN PHYTON

*"El talento gana partidos,
pero el trabajo en equipo y la inteligencia
ganan campeonatos"*
Michael Jordan

1.3. EL INTÉRPRETE

El intérprete de Python y la extensiva librería estándar se encuentran disponibles en la Web, <https://www.python.org/>, y se pueden distribuir. El mismo sitio contiene distribuciones y direcciones a muchos módulos de Python de terceras partes, programas, herramientas y adicionalmente documentación. Si ya sabes programar en C es fácil añadir nuevas funciones o módulos al intérprete, ya sea para realizar operaciones críticas a velocidad máxima, o para enlazar programas de Python con bibliotecas que tal vez sólo estén disponibles de forma binaria (por ejemplo, bibliotecas gráficas específicas de un fabricante).

El intérprete de python se suele instalar en `/usr/bin/python3.9`

Para arrancarlo, desde el terminal

```
python3.9
```

Con poner python3 es suficiente

```
rosa@rosa-Essential14L: ~  
rosa@rosa-Essential14L:~$ python3.9  
Python 3.9.2 (default, Feb 20 2021, 20:56:08)  
[GCC 9.3.0] on linux  
Type "help", "copyright", "credits" or "license" for more information.  
>>>  
rosa@rosa-Essential14L:~$
```

Se puede salir del intérprete Control + D o Control + Z. Si eso no funciona, puedes salir del intérprete escribiendo el comando: `quit()`.

Puedo escribir en el terminal las instrucciones de mi programa para que se vayan ejecutando

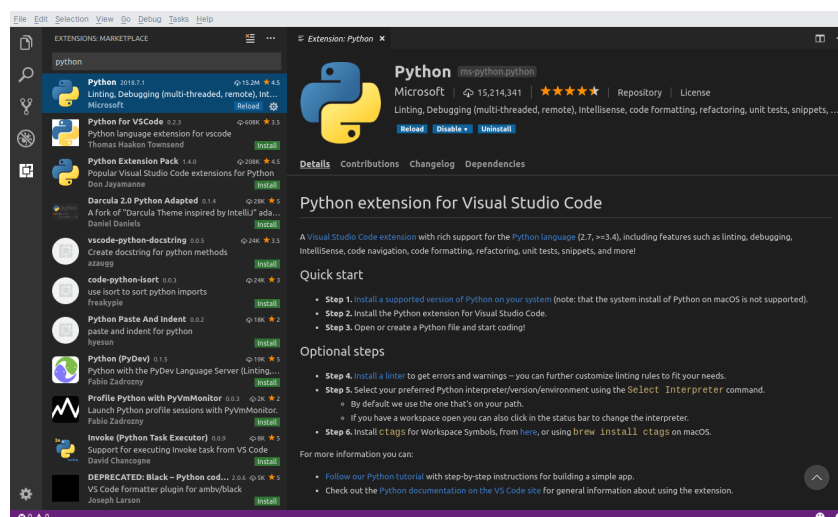
```
rosa@rosa-Essential14L:~$ python3.9
Python 3.9.2 (default, Feb 20 2021, 20:56:08)
[GCC 9.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> saludo="Hola Mundooooo"
>>> print(saludo)
Hola Mundooooo
>>> █
```

Pero lo mejor es crear un archivo de texto con extensión `.py` y después ejecutarlo. Se puede utilizar cualquier editor de código, nosotros usaremos Visual Studio Code porque actualmente es uno de los más utilizados. Otros ejemplos: Sublime Text, Brackets, Atom....

Antes de continuar, vamos a configurar el editor para programar en python:

1.Instalar la extensión de Python para Visual Studio Code

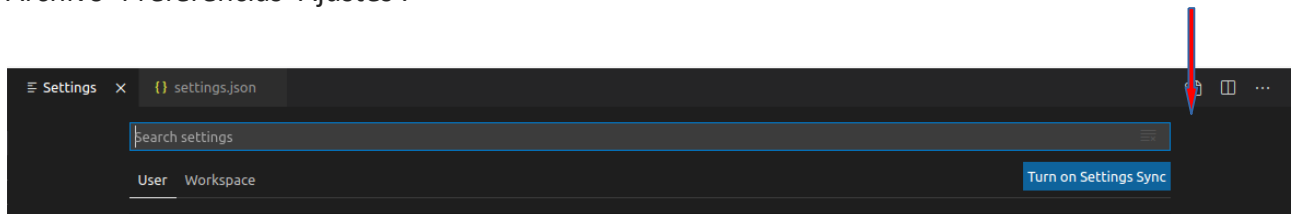
Lo primero que debemos hacer es instalar la extensión de Python para Visual Studio Code. Para ello, hacemos click en el menú de extensiones a la izquierda y tecleamos 'Python'. Seleccionamos la de Microsoft y la instalamos.



2. Configurar la extensión de Python en Visual Studio Code

Ahora que ya tenemos instalada la extensión de Python, podemos pasar a configurarla. Visual Studio Code gestiona las configuraciones mediante archivos JSON y se utilizan principalmente dos: un archivo para la configuración global y otro archivo específico para las configuraciones de proyectos.

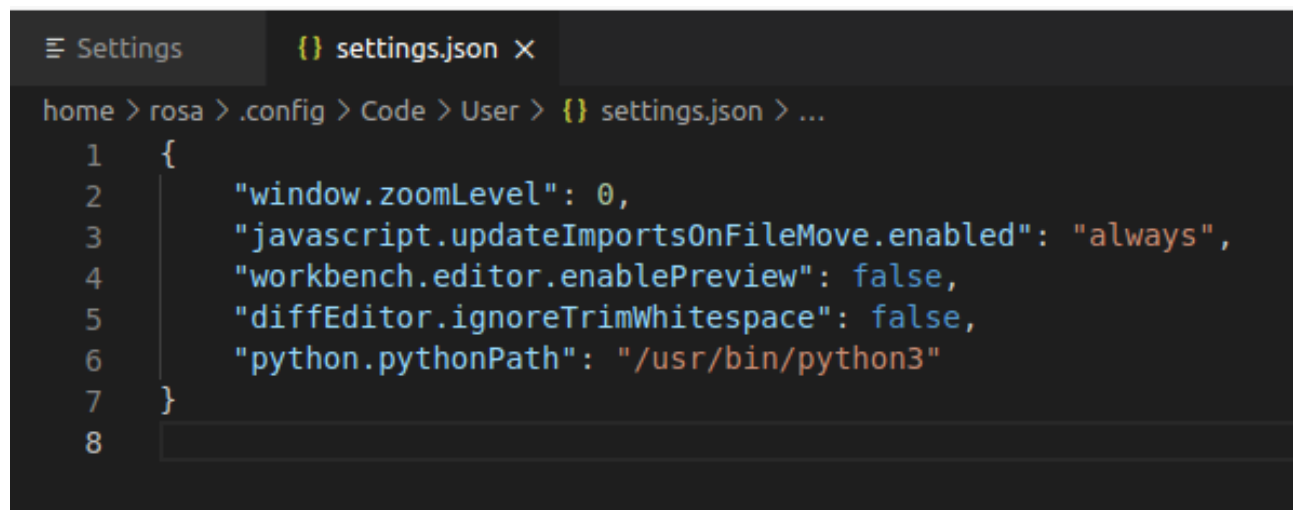
Para abrir la configuración global podemos utilizar el atajo Control+, (coma) o navegar por 'Archivo>Preferencias>Ajustes'.



Pulsar sobre el icono señalado en la imagen para abrir el archivo de configuración global para el usuario. Añadir el siguiente fragmento:

```
{  
  
  "python.pythonPath": "/usr/bin/python3"  
  
}
```

En este caso, añadimos /usr/bin/python3 porque es la ruta en la que se encuentra mi ejecutable de Python, pero vosotros tendréis que añadir la ruta en la que lo hayáis instalado.



Instalamos manualmente también el autopep8 para formatear los scripts

```
rosa@rosa-Essential14L:~/Escritorio/DJANGO$ pip install autopep8
Collecting autopep8
  Downloading autopep8-1.6.0-py2.py3-none-any.whl (45 kB)
    |████████████████████████████████████████| 45 kB 1.4 MB/s
Collecting pycodestyle>=2.8.0
  Downloading pycodestyle-2.8.0-py2.py3-none-any.whl (42 kB)
    |████████████████████████████████████████| 42 kB 1.7 MB/s
Collecting toml
  Downloading toml-0.10.2-py2.py3-none-any.whl (16 kB)
Installing collected packages: pycodestyle, toml, autopep8
Successfully installed autopep8-1.6.0 pycodestyle-2.8.0 toml-0.10.2
rosa@rosa-Essential14L:~/Escritorio/DJANGO$
```

3. Linters de código

Un linter de código es una herramienta que nos ayuda a seguir las buenas prácticas de código. Por ejemplo, si nos sobran espacios en blanco, si utilizamos una variable sin inicializar.... Python soporta varios linters de código entre los que se encuentran pep8, flake8 y pylint. El linter por defecto es pylint.

Para instalar pylint desde el terminal

```
sudo apt install pylint
```

Una vez instalado para usarlo

```
pylint archivo
```

Devuelve resultados que consisten en errores semánticos, errores de sintaxis, errores en el estilo de codificación, errores en el código, código excesivo y redundante, etc. También asigna una puntuación que indica si el código Python es ideal para usar y mantiene un historial de puntuaciones obtenidas al ejecutar un archivo de Python, así como después de cada edición.

```
e3uxFLcvWVDAf/1g_asu0r8sv9VuqnE2z8ReFXPo5xuHtrx$ pylint hola.py
***** Module hola
hola.py:1:0: C0114: Missing module docstring (missing-module-docstring)

-----
Your code has been rated at 0.00/10
```

El warning que nos da es porque aconseja incluir al principio del programa una “cadena de documentos”, por ejemplo, cómo funciona una clase, una función del programa....

```
1 """
2 Mi primer ejemplo
3 """
4 print("hola mundoooo")
```

Si dejamos más de una línea en blanco al final del programa nos dará un warning Trailing newlines (trailing-newlines)

En Visual Studio Code está instalado por defecto este linter, si no fuese así al escribir nuestro primer programa nos avisaría.

Otras extensiones útiles

1. [Python Docstring Generator](#)

Mantener una buena documentación en tu código es valioso y esta extensión te ayudará a cumplir con tu cometido. Pues al colocar comillas dobles dentro de una clase, método o función generará una plantilla para tu docstring y así proveer de la información necesaria de lo que estás creando.

2. [Code Runner](#)

Es natural que mientras escribimos código lo depuramos mediante su ejecución y para esto mismo es Code Runner. Podrá ejecutar tu código directamente desde la misma ventana sin tener que pasar a la terminal.

Este plugin es grandioso no solo para Python, pues es compatible con más de 20 lenguajes de programación distintos.

3. Trailing Spaces

Otro plugin de uso general es Trailing Spaces, el cual permite ver aquellos espacios vacíos al final o entre líneas para eliminarlos. Bastante práctico considerando que Python utiliza la indentación como base de su estructura.

4. Phyton Indent

Hablando de indentación, hay que estar muy atentos no solo a los espacios en blanco sino también a los ocupados. Por lo que a través de colores distintos esta extensión nos mostrará la “profundidad” que llevan nuestras líneas de código.

5. Better Comments

Al igual que los docstrings también podemos colocar comentarios que sean efectivos y con este plugin puedes utilizar un formato de color especial para datos importantes, alertas, tareas pendientes o consideraciones.

Better Comments es compatible también con múltiples lenguajes.

6. Djaneiro

Si trabajas con Django sabes lo importante que es ahorrar tiempo al escribir tu código y con esta extensión lo lograrás. Gracias a su colección de snippets podrás no solo ser más ágil, sino que también puedes contribuir al proyecto por su apertura.

7. Phyton extended

Ahora que si tu ambiente productivo es el de Flask, entonces puedes tomar la colección de snippets que tiene PyCharm para llevarlos a Visual Studio Code y mantener tu rendimiento con una escritura ágil.

Yo he instalado la 4 y la 7